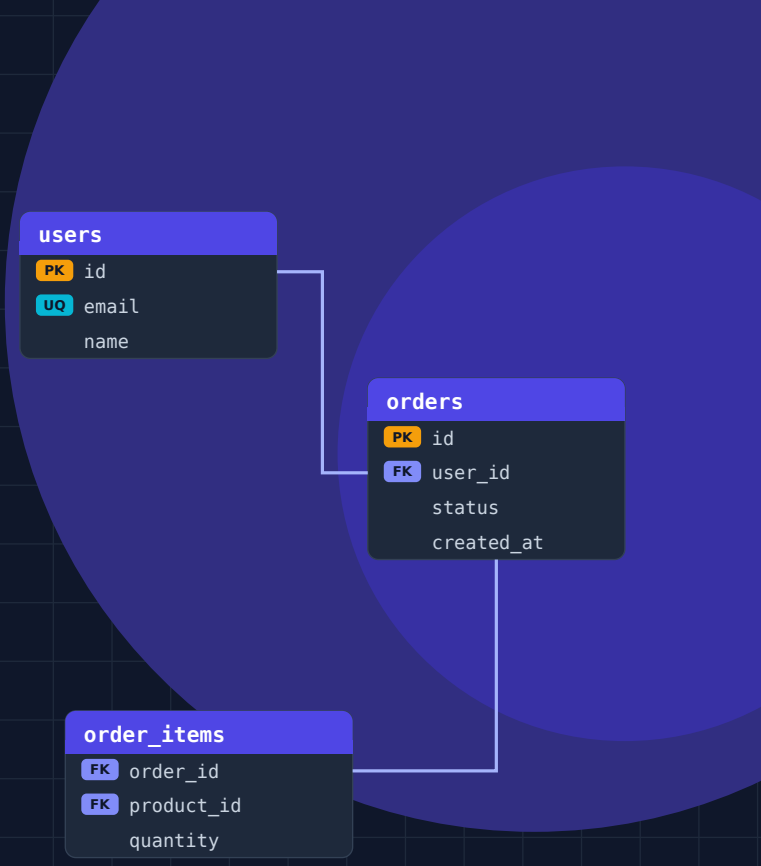


ПРАКТИЧЕСКИЙ ГАЙД

Проектирование баз данных

Связи, ключи, ограничения и принципы хорошей схемы — на понятных примерах

1 : 1 · 1 : N · N : 1 · M : N · PRIMARY KEY · FOREIGN KEY · CHECK · UNIQUE · Нормализация · Предметная область · Лайфхаки

Для начинающих и продолжающих

Примеры на PostgreSQL · Принципы подходят для любой реляционной СУБД

Содержание

1. Фундамент 5

Как устроена таблица 5

2. Ключи 6

Первичный ключ (PRIMARY KEY) 6

Натуральный или суррогатный ключ? 6

BIGINT или UUID? 7

Составной первичный ключ 7

Внешний ключ (FOREIGN KEY) 8

Что делать при удалении: ON DELETE 8

Уникальный ключ (UNIQUE) 9

3. Ограничения (constraints) 10

Все ограничения в одной таблице 10

NOT NULL — ограничение номер один 11

CHECK — бизнес-правила в схеме 11

Частичный и функциональный UNIQUE 12

EXCLUDE — для продолжающих 12

Проверять в коде или в базе? 12

4. Связи между таблицами 13

Как определить тип связи за 10 секунд 13

Связь один к одному (1 : 1) 14

Связь один ко многим (1 : N) 16

Связь многие к одному (N : 1) 18

Связь многие ко многим (M : N) 20

Сводка по связям 22

5. Нормализация без боли 23

Что не так с «одной большой таблицей» 23

Три нормальные формы простыми словами 23

Денормализация: когда можно нарушать 24

6. Как понять предметную область 25

Пошаговый процесс 25

Разбор на примере 26

Вопросы, которые стоит задавать всегда 26

Сущность или атрибут? 27

7. Практика: интернет-магазин 28

Полный SQL 29

Почему сделано именно так 30

Проверяем схему запросами 30

8. Типы данных и именование 31

Какой тип выбрать 31

Правила именования 32

9. Индексы: необходимый минимум 33

Где индексы появляются сами, а где нет 33

Составные индексы и правило «левого края» 33

Сколько индексов нужно 34

10. Лайфхаки и антипаттерны 35

Лайфхаки 35

Антипаттерны 36

11. Чек-лист и шпаргалка 38

Чек-лист схемы 38

Шпаргалка на одну страницу 39

Перед стартом

База данных живёт дольше любого кода. Приложение переписывают, фреймворки меняются, а таблицы с данными клиентов остаются годами. Поэтому схема — это не техническая мелочь, а **фундамент**: если он кривой, всё, что сверху, будет постоянно трескаться.

Этот гайд — про то, как строить этот фундамент правильно. Принципы здесь одни и те же для новичка и для опытного разработчика: просто опытный применяет их на автомате. Примеры написаны на **PostgreSQL** (все они проверены на реальной базе), но идеи работают в MySQL, SQLite, SQL Server и любой другой реляционной СУБД.

Что внутри

- **Ключи** — как база отличает одну строку от другой и связывает таблицы.
- **Ограничения** — как сделать так, чтобы неправильные данные просто не могли попасть в базу.
- **Связи 1:1, 1:N, N:1, M:N** — с диаграммами, SQL, примерами данных и запросами.
- **Нормализация** — без занудства, на одном наглядном примере.
- **Как понять предметную область** — пошаговый процесс от разговора к схеме.
- **Практика** — полная схема интернет-магазина с объяснением каждого решения.
- **Типы данных, именование, индексы, лайфхаки, антипаттерны** и чек-лист.

Обозначения в гайде

> **→ Правило**
> Короткое правило, которое стоит запомнить.

> **✓ Лайфхак**
> Приём из практики, который экономит время и нервы.

> **⚠ Осторожно**
> Место, где легко ошибиться или выстрелить себе в ногу.

> **✗ Ошибка новичка**
> Типичная ошибка новичка — и как сделать правильно.

> **★ Важно понять**
> Ключевая идея, на которой держится всё остальное.

01 Фундамент

Словарь, без которого дальше никак

Прежде чем рисовать связи, договоримся о словах. Многие путаются не потому, что тема сложная, а потому что одни и те же вещи называют по-разному.

Термин	Что это простыми словами	Пример
Сущность (entity)	«Штука» из реального мира, о которой нужно хранить данные	Пользователь, Заказ, Товар
Таблица	Место, где хранятся все экземпляры одной сущности	`users`, `orders`
Строка (запись)	Один конкретный экземпляр сущности	Пользователь Анна
Столбец (атрибут)	Одно свойство сущности, у каждого есть тип	`email TEXT`
Схема	Структура всей базы: таблицы, столбцы, связи и правила	ER-диаграмма + SQL
Связь (relationship)	Как экземпляры разных сущностей относятся друг к другу	Пользователь *делает* заказы
Кардинальность	Сколько экземпляров может быть с каждой стороны связи	1:1, 1:N, M:N
Ключ	Столбец (или набор), по которому находят строку или ссылаются на неё	`id`, `user_id`
Ограничение (constraint)	Правило, нарушить которое база не позволит	`price >= 0`

★ Важно понять

Хорошая схема делает **неправильные данные невозможными**. Не «маловероятными», не «проверяемыми в коде», а именно невозможными: база сама откажется сохранить заказ без покупателя, товар с отрицательной ценой или двух пользователей с одним email.

Всё, что дальше в гайде, — инструменты для достижения этой цели.

Как устроена таблица

Таблица — это сетка: столбцы задают *структуру* (что мы храним), строки — *данные* (конкретные значения). Порядок строк в реляционной базе не гарантирован: если нужен порядок — всегда пиши `ORDER BY`.

users

id	email	name	created_at
1	anna@mail.ru	Анна	2026-01-10 09:15
2	boris@mail.ru	Борис	2026-02-03 18:40
3	vera@mail.ru	Вера	2026-03-21 12:05

Таблица users: 4 столбца (структура) и 3 строки (данные)

02

Ключи

Первичные, внешние, уникальные, составные

Первичный ключ (PRIMARY KEY)

Первичный ключ — это «паспорт» строки. По нему база однозначно находит запись, и на него ссылаются другие таблицы. У таблицы может быть **только один** первичный ключ (но он может состоять из нескольких столбцов).

Четыре требования к хорошему первичному ключу:

1. **Уникальный** — двух строк с одинаковым ключом быть не может.
2. **Никогда не NULL** — у каждой строки ключ есть всегда.
3. **Стабильный** — не меняется со временем. Если ключ поменяется, придётся обновлять все ссылки на него во всех таблицах.
4. **Минимальный** — без лишних столбцов: если можно обойтись одним, не надо брать два.

Классический первичный ключ

```
CREATE TABLE users (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email      TEXT        NOT NULL UNIQUE,
    name       TEXT        NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

`GENERATED ALWAYS AS IDENTITY` — современный стандартный способ сделать автоинкремент: база сама выдаёт 1, 2, 3… `PRIMARY KEY` автоматически означает `NOT NULL` + `UNIQUE` + индекс.

> **✔ Лайфхак**
>
> Встречал `SERIAL` в старых туториалах? Это устаревший способ. В новых проектах на PostgreSQL используй `GENERATED ALWAYS AS IDENTITY` — он соответствует стандарту SQL и защищает от случайной ручной вставки id.

Натуральный или суррогатный ключ?

Натуральный ключ — данные из реального мира, которые и так уникальны: email, номер паспорта, ИНН, артикул. **Суррогатный ключ** — искусственный идентификатор без смысла (`id = 1, 2, 3…` или UUID), который нужен только базе.

	Натуральный (email, ИНН)	Суррогатный (id)
Смысл	Есть, понятен человеку	Нет, просто номер
Стабильность	Ненадёжная: email меняют, паспорт меняют, в ИНН бывают ошибки ввода	Полная: никогда не меняется
Размер	Часто длинная строка — тяжелее индексы и ссылки	8 байт (BIGINT) или 16 (UUID)

	Натуральный (email, ИНН)	Суррогатный (id)
Утечка данных	Email в URL и в логах — так себе идея	Ничего не раскрывает
Когда брать	Редко: коды стран (`RU`), валют (`USD`) — то, что не меняется десятилетиями	Почти всегда

→ Правило

Суррогатный ключ как PRIMARY KEY + UNIQUE на натуральный ключ. Так ты получаешь стабильные ссылки между таблицами и одновременно гарантию, что email не повторится. Лучшее от обоих миров.

BIGINT или UUID?

	BIGINT (IDENTITY)	UUID
Размер	8 байт	16 байт
Скорость	Максимальная, индекс компактный	Чуть медленнее; случайный UUIDv4 «размазывает» вставки по индексу
Кто генерирует	Только база	Можно сгенерировать в приложении заранее
Предсказуемость	`/users/41`, `/users/42` — легко перебрать и узнать объём бизнеса	Угадать невозможно
Удобство	Легко читать и диктовать	Длинный, неудобно глазами

✔ Лайфхак

Нужен UUID — бери **UUIDv7**: он упорядочен по времени, поэтому индекс не страдает. В PostgreSQL 18+ есть встроенная функция `uuidv7()`, в более старых версиях генерируй его в приложении. Для внутренних таблиц смело используй `BIGINT` — это не ошибка, а нормальный выбор.

Составной первичный ключ

Иногда строку однозначно определяет не один столбец, а **комбинация**. Типичный случай — связующие таблицы (подробнее в главе про M:N): пара «товар + тег» уникальна сама по себе, отдельный `id` ей не нужен.

```
CREATE TABLE product_tags (
    product_id BIGINT NOT NULL,
    tag_id     BIGINT NOT NULL,
    PRIMARY KEY (product_id, tag_id)   -- пара уникальна, повторов не будет
);
```

Внешний ключ (FOREIGN KEY)

Внешний ключ — это столбец, значение которого **обязано существовать** в другой таблице. Он и превращает набор отдельных таблиц в связанную базу данных. Термин для этой гарантии — *ссылочная целостность*.

Внешний ключ в действии

```
CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Попытка сослаться на несуществующего пользователя:
INSERT INTO orders (user_id) VALUES (999);
-- ERROR: insert or update on table "orders" violates foreign key
--        constraint "orders_user_id_fkey"
--        DETAIL: Key (user_id)=(999) is not present in table "users".
```

Что внешний ключ делает за тебя:

- Не даёт создать заказ для несуществующего пользователя.
- Не даёт удалить пользователя, у которого есть заказы (если не сказано иное — см. ниже).
- Документирует схему: глядя на `REFERENCES users (id)`, любой поймёт, как связаны таблицы.

> **✗ Ошибка новичка**
>
> «Внешние ключи тормозят, мы проверяем всё в коде». Проверка в коде **не спасает** от гонок (два запроса одновременно), ручных правок в консоли, скриптов миграции и багов. Через полгода в базе обязательно появятся «осиротевшие» строки, ссылающиеся в никуда. Цена внешнего ключа ничтожна по сравнению с ценой мусорных данных.

Что делать при удалении: ON DELETE

Когда удаляют строку-«родителя», база должна знать, что делать с «детьми», которые на неё ссылаются. Это задаётся в определении внешнего ключа.

Действие	Что происходит	Когда использовать
`NO ACTION` **(по умолчанию)**	Удаление запрещено, если есть дети	Безопасный дефолт. Почти всегда хороший выбор
`RESTRICT`	То же самое, но проверка мгновенная (нельзя отложить до конца транзакции)	Когда хочешь подчеркнуть: «удалять нельзя никогда»
`CASCADE`	Дети удаляются вместе с родителем	Только для «частей целого»: позиции заказа, ответы на опрос
`SET NULL`	У детей ссылка становится NULL	Связь необязательная: менеджер уволился — у клиента нет менеджера
`SET DEFAULT`	У детей ссылка становится значением по умолчанию	Редко

Примеры выбора действия

```
-- Позиции заказа — часть заказа: удалили заказ -> удалились позиции
order_id BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE

-- Категорию с товарами удалять нельзя
category_id BIGINT NOT NULL REFERENCES categories (id) ON DELETE RESTRICT

-- Менеджер уволился -> у клиента просто нет менеджера
manager_id BIGINT REFERENCES employees (id) ON DELETE SET NULL
```

⚠ Осторожно

`CASCADE` — мощный и опасный инструмент. Один `DELETE` может по цепочке снести тысячи строк в пяти таблицах. Задай себе вопрос: «Имеет ли ребёнок смысл без родителя?» Позиция заказа без заказа — нет, `CASCADE` уместен. Заказы без пользователя — да, это история продаж и бухгалтерия: тут `RESTRICT`, а пользователя лучше «мягко» удалить.

★ Важно понять

PostgreSQL не создаёт индекс на столбец внешнего ключа автоматически. Индекс есть только на стороне `PRIMARY KEY`. Без индекса на `orders.user_id` каждый запрос «заказы пользователя» и каждое удаление пользователя будут сканировать всю таблицу заказов. Правило: **создал FK — создай индекс**. (MySQL InnoDB, кстати, создаёт его сам.)

Уникальный ключ (UNIQUE)

Кроме первичного ключа в таблице часто есть другие столбцы, которые обязаны быть уникальными: email, артикул, slug для URL, номер телефона. Их называют *кандидатными* или *альтернативными* ключами и защищают ограничением `UNIQUE`.

	PRIMARY KEY	UNIQUE
Сколько на таблицу	Ровно один	Сколько угодно
NULL	Запрещён	Разрешён (и несколько NULL не считаются дублями)
Назначение	Идентифицировать строку и быть целью для FK	Гарантировать бизнес-правило «не повторяется»

03 Ограничения (constraints)

Правила, которые база соблюдает за тебя

Ограничение — это правило, встроенное прямо в схему. Если данные его нарушают, база откажется их сохранить и вернёт ошибку. Это самый дешёвый и надёжный способ держать данные чистыми.

Ограничение	Что гарантирует	Пример
`NOT NULL`	Значение обязательно есть	`name TEXT NOT NULL`
`UNIQUE`	Значение не повторяется	`email TEXT UNIQUE`
`PRIMARY KEY`	`NOT NULL` + `UNIQUE`, идентификатор строки	`id BIGINT PRIMARY KEY`
`FOREIGN KEY`	Значение существует в другой таблице	`REFERENCES users (id)`
`CHECK`	Произвольное условие истинно	`CHECK (price >= 0)`
`DEFAULT`	Значение, если его не указали (формально не ограничение, но рядом)	`DEFAULT now()`
`EXCLUDE`	Строки не «конфликтуют» (только PostgreSQL)	Брони не пересекаются по времени

Все ограничения в одной таблице

Таблица товаров с именованными ограничениями

```
CREATE TABLE products (
    id          BIGINT      GENERATED ALWAYS AS IDENTITY,
    sku         TEXT        NOT NULL,
    name        TEXT        NOT NULL,
    price_cents BIGINT      NOT NULL,
    stock       INTEGER     NOT NULL DEFAULT 0,
    status      TEXT        NOT NULL DEFAULT 'draft',
    category_id BIGINT      NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT products_pk          PRIMARY KEY (id),
    CONSTRAINT products_sku_uq      UNIQUE (sku),
    CONSTRAINT products_price_chk   CHECK (price_cents >= 0),
    CONSTRAINT products_stock_chk   CHECK (stock >= 0),
    CONSTRAINT products_status_chk
        CHECK (status IN ('draft', 'active', 'archived')),
    CONSTRAINT products_category_fk
        FOREIGN KEY (category_id) REFERENCES categories (id)
        ON DELETE RESTRICT
);
```

Обрати внимание: каждое ограничение получило **имя** через `CONSTRAINT имя`. Это не обязательно (база придумает имя сама), но очень полезно:

```
INSERT INTO products (sku, name, price_cents, category_id)
VALUES ('KB-01', 'Клавиатура', -500, 1);
-- ERROR: new row for relation "products" violates check constraint
--        "products_price_chk"
-- По имени ограничения сразу понятно, что не так.
```

- **Понятные ошибки.** По имени `products_price_chk` в логах сразу ясно, что пошло не так. В приложении можно поймать ошибку по имени и показать пользователю человеческое сообщение.
- **Простые миграции.** `ALTER TABLE products DROP CONSTRAINT products_price_chk` — и не нужно гадать, как база назвала его автоматически.
- **Единый стиль.** Шаблон `таблица_суть_тип`: `_pk`, `_fk`, `_uq`, `_chk`.

NOT NULL — ограничение номер один

`NULL` в SQL означает «значение неизвестно», а не «ноль» и не «пустая строка». Из-за этого у него странная логика, которая ломает интуицию:

Выражение	Результат	Почему
`NULL = NULL`	`NULL` (не `true`!)	Неизвестное не равно неизвестному
`WHERE x <> 5`	Не вернёт строки, где x IS NULL	Сравнение с NULL — не true
`COUNT(x)`	Не считает NULL	В отличие от `COUNT(*)`
`10 + NULL`	`NULL`	Любая арифметика с NULL даёт NULL
Проверка	`x IS NULL` / `x IS NOT NULL`	Только так, не через `=`

> **→ Правило**
>
> **По умолчанию каждый столбец — NOT NULL.** Разрешай NULL только тогда, когда отсутствие значения реально что-то означает: «дата отправки ещё неизвестна», «исполнитель ещё не назначен». Каждый nullable столбец — это ветка `if` в коде и потенциальный баг.

CHECK — бизнес-правила в схеме

`CHECK` может проверять не только один столбец, но и соотношение между несколькими. Всё, что можно выразить условием на одной строке, лучше закрепить здесь.

```
CREATE TABLE promotions (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    starts_at      TIMESTAMPTZ NOT NULL,
    ends_at        TIMESTAMPTZ NOT NULL,
    discount_pct   SMALLINT    NOT NULL,

    CONSTRAINT promotions_dates_chk    CHECK (ends_at > starts_at),
    CONSTRAINT promotions_discount_chk CHECK (discount_pct BETWEEN 1 AND 90)
);
```

Хорошие кандидаты в `CHECK`: неотрицательные цены и остатки, рейтинг от 1 до 5, дата окончания позже даты начала, статус из фиксированного списка, процент от 0 до 100, длина строки (`CHECK (length(code) = 6)`).

Частичный и функциональный UNIQUE

Две частые проблемы: `Anna@Mail.ru` и `anna@mail.ru` — это технически разные строки, а при «мягком удалении» (`deleted_at`) удалённый пользователь блокирует повторную регистрацию с тем же email. Обе решаются уникальным индексом:

```
-- Email уникален без учёта регистра и только среди "живых" пользователей
CREATE UNIQUE INDEX users_email_active_uq
    ON users (lower(email))
    WHERE deleted_at IS NULL;
```

✓ Лайфхак

Уникальный индекс по выражению (`lower(email)`) и с условием (`WHERE deleted_at IS NULL`) — одна из самых полезных и недооценённых возможностей PostgreSQL. В MySQL похожего добиваются через генерируемые столбцы.

EXCLUDE — для продолжающих

Как запретить двойную бронь переговорки на одно время? `UNIQUE` тут не поможет: брони не равны, они *пересекаются*. В PostgreSQL для этого есть `EXCLUDE`: «не может быть двух строк, у которых `room_id` равен **и** периоды пересекаются (оператор `&&`)».

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE rooms (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE bookings (
    id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id BIGINT    NOT NULL REFERENCES rooms (id),
    period  TSTZRANGE NOT NULL,
    -- одна комната не может быть забронирована на пересекающееся время
    CONSTRAINT bookings_no_overlap
        EXCLUDE USING gist (room_id WITH =, period WITH &&)
);
```

Проверять в коде или в базе?

★ Важно понять

В обоих местах, но с разными целями.

Код проверяет, чтобы показать пользователю понятное сообщение («Email уже занят») до похода в базу. Это вопрос удобства.

База проверяет, чтобы гарантировать целостность. Это вопрос корректности. Классический пример: два запроса одновременно проверяют, свободен ли email — оба видят «свободен», оба вставляют. Спасёт только `UNIQUE` в базе.

Если приходится выбирать одно — выбирай базу.

04 Связи между таблицами

1:1, 1:N, N:1, M:N — как определить и как реализовать

Как определить тип связи за 10 секунд

Возьми две сущности A и B и задай **два вопроса** — с каждой стороны:

1 **Сколько B может быть у одного A?**
Например: сколько заказов может быть у одного пользователя? — *Много.*

2 **Сколько A может быть у одного B?**
Сколько пользователей может быть у одного заказа? — *Один.*

Ответ с одной стороны	Ответ с другой	Тип связи	Как реализовать
Один	Один	**1 : 1**	FK + UNIQUE (или общий PK)
Много	Один	**1 : N** (он же N : 1)	FK на стороне «много»
Много	Много	**M : N**	Третья (связующая) таблица

→ Правило

Внешний ключ всегда живёт на стороне «много». У заказа один пользователь — значит, `user_id` хранится в заказе. У пользователя много заказов — хранить их список в пользователе нельзя (в одной ячейке — одно значение). Если «много» с обеих сторон, FK некуда положить — поэтому появляется третья таблица.

Как читать диаграммы в этом гайде

Мы используем нотацию «воронья лапка» (crow's foot) — её понимают все инструменты и все разработчики. Значок рисуется у той таблицы, о количестве строк которой идёт речь. Кружок = «может не быть», черта = «обязательно».



ровно один
`1`



ноль или один
`0..1`



один или много
`1..N`

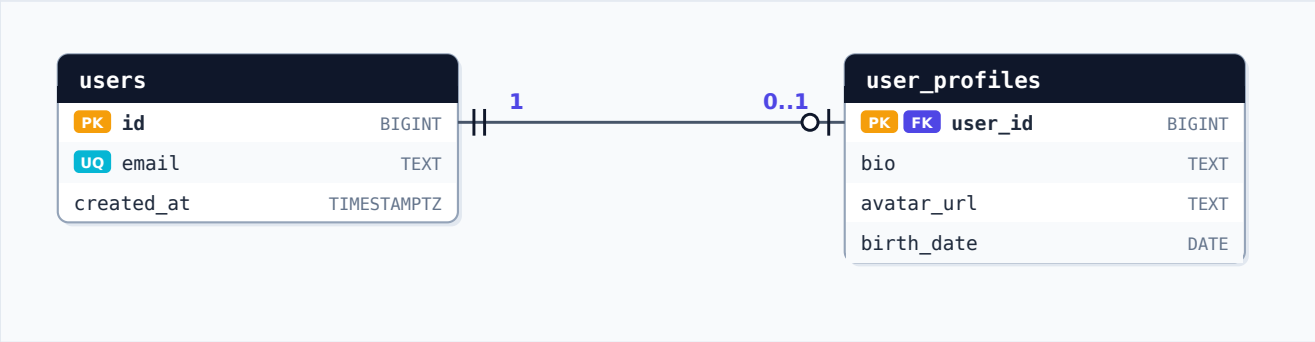


ноль или много
`0..N`

Метка	Значение
`PK`	Первичный ключ
`FK`	Внешний ключ
`UQ`	Уникальное значение

Связь один к одному (1 : 1)

Одной строке в A соответствует **не больше одной** строки в B, и наоборот. Пример: пользователь и его профиль. У пользователя максимум один профиль, профиль принадлежит ровно одному пользователю.



У пользователя может быть ноль или один профиль; профиль всегда принадлежит ровно одному пользователю

Два способа реализации. Главное в обоих — гарантировать, что на одного пользователя не появится второй профиль.

```
CREATE TABLE users (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email      TEXT        NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
-- Способ А (рекомендую): общий первичный ключ
CREATE TABLE user_profiles (
    user_id    BIGINT PRIMARY KEY
               REFERENCES users (id) ON DELETE CASCADE,
    bio        TEXT,
    avatar_url TEXT,
    birth_date DATE
);
```

```
-- Способ Б: свой id + внешний ключ с UNIQUE
CREATE TABLE user_profiles (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL UNIQUE      -- UNIQUE делает связь 1:1
               REFERENCES users (id) ON DELETE CASCADE,
    bio        TEXT,
    avatar_url TEXT,
    birth_date DATE
);
```

✗ Ошибка новичка

Забыть `UNIQUE` на `user_id` в способе Б. Без него ничто не мешает вставить второй профиль тому же пользователю — и твоя связь 1:1 тихо превращается в 1:N. Схема должна **гарантировать** кардинальность, а не надеяться на аккуратность кода.

В способе А `user_id` одновременно и первичный, и внешний ключ: уникальность получается автоматически, лишнего столбца нет. Поэтому он проще и строже.

Пример данных

users

id	email
1	anna@mail.ru
2	boris@mail.ru
3	vera@mail.ru

user_profiles

user_id	bio	birth_date
1	Люблю SQL	1995-04-12
3	Дизайнер	1998-11-02

У Бориса (id = 2) профиля нет — это нормально, связь «ноль или один»

```
SELECT u.email, p.bio, p.birth_date
FROM users u
LEFT JOIN user_profiles p ON p.user_id = u.id;
-- LEFT JOIN, потому что профиля может не быть
```

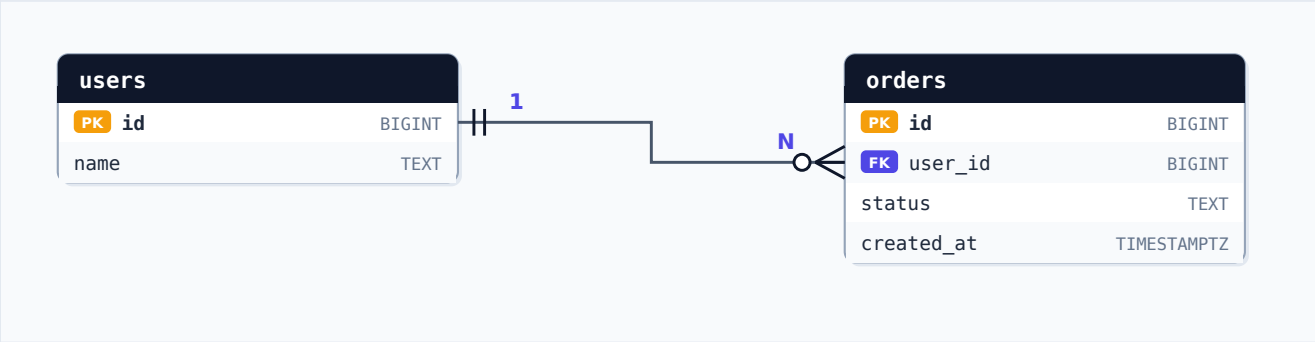
Когда 1:1 действительно нужна

- **Необязательные данные.** Профиль заполняют 20% пользователей — незачем держать десяток пустых столбцов в основной таблице.
- **Чувствительные данные.** Паспорт, медицинские данные, платёжная информация — в отдельной таблице с отдельными правами доступа.
- **Тяжёлые и редко нужные данные.** Большие тексты или JSON, которые не нужны в 95% запросов к основной таблице.
- **Разный жизненный цикл.** Данные появляются позже (верификация личности после регистрации) или меняются гораздо чаще.
- **Подтипы.** Общая таблица `payments` и детали в `card_payments` / `bank_transfers`.

> ⚠ **Осторожно**
>
> Если данные **всегда** есть и **всегда** нужны вместе — это не 1:1, это просто столбцы одной таблицы. Связь 1:1 «для красоты» добавляет лишний JOIN в каждый запрос и ничего не даёт взамен.

Связь один ко многим (1 : N)

Самая частая связь в любой базе — их будет 80% от всех. Одной строке в A соответствует **сколько угодно** строк в B, но каждая строка B принадлежит **одной** строке A. Пример: у пользователя много заказов, у заказа один пользователь.



Один пользователь — ноль или много заказов. FK user_id — на стороне «много»

```sql
CREATE TABLE users (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL
);
```

```sql
CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id) ON DELETE RESTRICT,
    status     TEXT   NOT NULL DEFAULT 'new',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- PostgreSQL НЕ создаёт индекс на внешний ключ сам — делаем руками
CREATE INDEX orders_user_id_idx ON orders (user_id);
```

Пример данных

users

id	name
1	Анна
2	Борис
3	Вера

orders

id	user_id	status
101	1	paid
102	1	new
103	2	shipped

У Анны два заказа, у Бориса один, у Веры ни одного. Каждая строка orders «показывает» на своего владельца

```sql
-- Сколько заказов у каждого пользователя (включая тех, у кого 0)
SELECT u.name, COUNT(o.id) AS orders_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name
ORDER BY orders_count DESC;
```

✗ Ошибка новичка

Хранить «многое» внутри «одного»:

```
-- ✗ Плохо: список заказов внутри пользователя
CREATE TABLE users (
    id        BIGINT PRIMARY KEY,
    name      TEXT,
    order_ids TEXT          -- '101,102,105'
);
```

Такой столбец нельзя проверить внешним ключом, по нему неудобно и медленно искать («у кого есть заказ 102?»), его нельзя нормально обновить без гонок, а удаление заказа оставит мусор в строке. Правильный вариант — тот, что выше: `user_id` в каждом заказе.

Обязательная и необязательная связь

Обязательность связи задаётся одним словом — `NOT NULL` у внешнего ключа. Заказ без покупателя не имеет смысла — ставим `NOT NULL`. Задача без исполнителя вполне нормальна — оставляем столбец nullable.

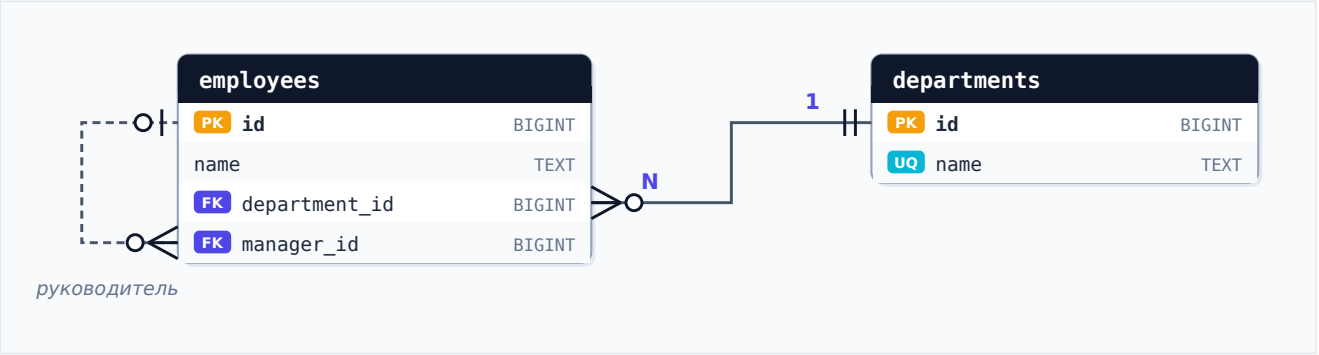
```
CREATE TABLE tasks (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title       TEXT   NOT NULL,
    author_id   BIGINT NOT NULL REFERENCES users (id),   -- автор есть всегда
    assignee_id BIGINT          REFERENCES users (id)    -- исполнителя может не быть
                ON DELETE SET NULL
);
```

Заметь: в `tasks` две связи с одной и той же таблицей `users`, у каждой своя роль. Это нормально — называй столбцы по роли (`author_id`, `assignee_id`), а не `user_id_1`, `user_id_2`.

Связь многие к одному (N : 1)

> **★ Важно понять**
>
> **N:1 — это та же самая связь, что и 1:N, просто прочитанная с другой стороны.** «У пользователя много заказов» (1:N) и «много заказов принадлежат одному пользователю» (N:1) описывают одни и те же две таблицы с одним и тем же внешним ключом. В SQL разницы нет никакой.

Зачем тогда отдельное название? Оно отражает **точку зрения**. Когда ты проектируешь таблицу `employees`, ты думаешь: «каждый сотрудник относится к **одному** отделу» — это N:1 от сотрудника. В ORM и коде это часто видно как пара *belongs to* (N:1) и *has many* (1:N). Полезно уметь смотреть с обеих сторон.



Много сотрудников → один отдел. Пунктир — связь таблицы с самой собой: много подчинённых → один руководитель

```sql
CREATE TABLE departments (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE employees (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name          TEXT   NOT NULL,
    department_id BIGINT NOT NULL REFERENCES departments (id),
    manager_id    BIGINT REFERENCES employees (id) ON DELETE SET NULL,

    CONSTRAINT employees_not_own_manager CHECK (manager_id <> id)
);

CREATE INDEX employees_department_id_idx ON employees (department_id);
CREATE INDEX employees_manager_id_idx    ON employees (manager_id);
```

Самоссылающаяся связь

`manager_id` ссылается на ту же таблицу `employees`: руководитель — тоже сотрудник. Это всё ещё обычная N:1 (много подчинённых → один руководитель), просто обе стороны в одной таблице. Так же моделируют дерево категорий (`parent_id`), ветки комментариев, структуру папок.

departments

id	name
1	Разработка
2	Продажи

employees

id	name	department_id	manager_id
1	Ольга	1	NULL
2	Игорь	1	1
3	Мария	1	1
4	Пётр	2	NULL

Ольга руководит Игорем и Марией; у Ольги и Петра руководителя нет

```sql
-- Сотрудник, его отдел и его руководитель
SELECT e.name  AS employee,
       d.name  AS department,
       m.name  AS manager
FROM employees e
JOIN departments d ON d.id = e.department_id
LEFT JOIN employees m ON m.id = e.manager_id;
```

Для продолжающих: обойти дерево любой глубины можно рекурсивным запросом.

```sql
-- Все подчинённые Ольги (id = 1) на всех уровнях
WITH RECURSIVE team AS (
    SELECT id, name, manager_id
    FROM employees
    WHERE id = 1

    UNION ALL

    SELECT e.id, e.name, e.manager_id
    FROM employees e
    JOIN team t ON e.manager_id = t.id
)
SELECT id, name FROM team;
```

Справочники — частный случай N:1

Много товаров → одна категория, много пользователей → одна страна, много заказов → один статус. Для «списка допустимых значений» есть три способа:

Способ	Плюсы	Минусы	Когда
`CHECK (status IN (...))`	Просто, видно в схеме	Новое значение = миграция	Короткий стабильный список
`ENUM` тип	Компактно, строгая типизация	Удалить или переименовать значение трудно	Редко меняется, не нужны доп. поля
Таблица-справочник + FK	Можно добавить поля (название, порядок, цвет), менять без миграции	Лишний JOIN	Список растёт, есть атрибуты, им управляют пользователи

Справочник с естественным текстовым ключом

```sql
CREATE TABLE order_statuses (
    code  TEXT PRIMARY KEY,          -- 'new', 'paid', 'shipped'
    title TEXT NOT NULL              -- 'Новый', 'Оплачен', 'Отправлен'
);

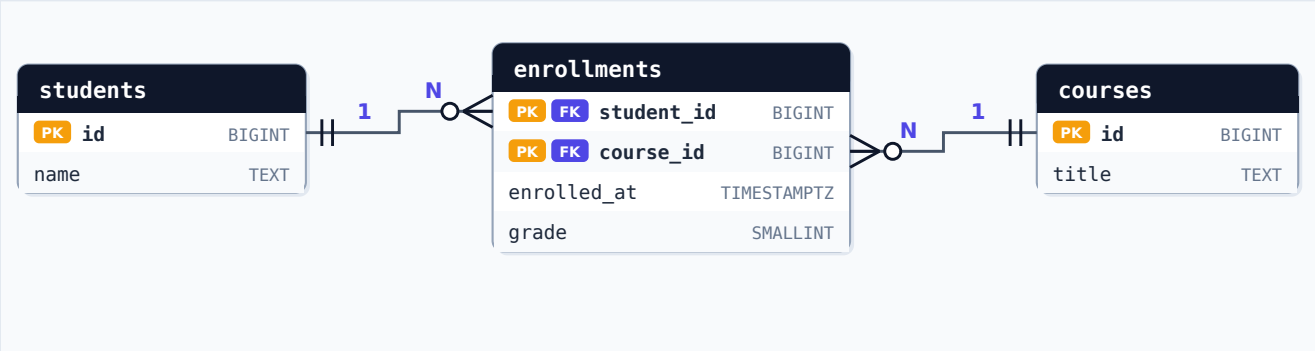
CREATE TABLE shipments (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    status_code TEXT NOT NULL REFERENCES order_statuses (code)
);
```

> **✔ Лайфхак**
>
> Для справочников натуральный ключ вроде `'paid'` — как раз тот редкий случай, когда он уместен: код стабилен и читается в данных без JOIN.

Связь многие ко многим (M : N)

Одной строке A соответствует много строк B, **и** одной строке B — много строк A. Пример: студент записан на много курсов, на курсе учится много студентов. Другие примеры: статьи и теги, фильмы и актёры, пользователи и роли, товары и заказы.

Положить внешний ключ некуда: ни `course_id` в студенте (курсов много), ни `student_id` в курсе (студентов много). Поэтому связь M:N **всегда** раскладывается на две связи 1:N через третью — **связующую** таблицу (junction table).



M:N = две связи 1:N. Каждая строка enrollments — один факт «такой-то студент записан на такой-то курс»

```sql
CREATE TABLE students (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE courses (
    id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title TEXT NOT NULL
);

-- Связующая таблица: одна строка = один студент на одном курсе
CREATE TABLE enrollments (
    student_id  BIGINT NOT NULL REFERENCES students (id) ON DELETE CASCADE,
    course_id   BIGINT NOT NULL REFERENCES courses (id)  ON DELETE CASCADE,
    enrolled_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    grade       SMALLINT CHECK (grade BETWEEN 1 AND 5),

    PRIMARY KEY (student_id, course_id)
);

-- PK (student_id, course_id) ускоряет поиск по student_id,
-- а для поиска по course_id нужен свой индекс
CREATE INDEX enrollments_course_id_idx ON enrollments (course_id);
```

Пример данных

students

id	name
1	Анна
2	Борис

enrollments

student_id	course_id	grade
1	10	5
1	20	4
2	10	NULL

courses

id	title
10	SQL
20	Алгоритмы
30	Сети

Анна на двух курсах, Борис на одном; на курсе SQL двое; на «Сети» пока никто не записан

```sql
-- Кто на какие курсы записан
SELECT s.name, c.title, e.grade
FROM students s
JOIN enrollments e ON e.student_id = s.id
JOIN courses     c ON c.id = e.course_id
ORDER BY s.name;

-- Сколько студентов на каждом курсе
SELECT c.title, COUNT(e.student_id) AS students
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title;
```

★ Важно понять

Связующая таблица часто оказывается полноценной сущностью. У записи на курс есть дата и оценка, у позиции заказа — количество и цена, у участия в проекте — роль и дата вступления. Если у связи есть собственные атрибуты — это сигнал, что ты нашёл новую сущность предметной области.

Поэтому называй её бизнес-термином, если он есть: `enrollments`, `order_items`, `memberships` вместо безликих `students_courses`.

Составной PK или собственный id?

	PRIMARY KEY (a_id, b_id)	Свой id + UNIQUE
Дубли пары	Невозможны автоматически	Нужно не забыть `UNIQUE (a_id, b_id)`
Ссылки на строку	Неудобно: другим таблицам придётся хранить два столбца	Удобно: одна ссылка `enrollment_id`
Когда брать	Чистая связь, на неё никто не ссылается	На связь будут ссылаться или пара может повторяться

Если одна и та же пара **может повторяться** (студент пересдаёт экзамен), значит уникальна не пара, а пара + что-то ещё:

```sql
-- Если одна и та же пара может повторяться (пересдачи),
-- у строки связи появляется свой id и номер попытки
CREATE TABLE exam_attempts (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_id BIGINT   NOT NULL REFERENCES students (id),
    course_id  BIGINT   NOT NULL REFERENCES courses (id),
    attempt_no SMALLINT NOT NULL CHECK (attempt_no > 0),
    score      SMALLINT NOT NULL,

    CONSTRAINT exam_attempts_uq UNIQUE (student_id, course_id, attempt_no)
);
```

✔ Лайфхак

Составной PK `(student_id, course_id)` ускоряет поиск по `student_id` (первый столбец индекса), но **не** по `course_id`. Поэтому в связующей таблице почти всегда нужен второй индекс — по второму столбцу.

Сводка по связям

Связь	Пример	Где FK	Ключевое ограничение
1 : 1	Пользователь — профиль	В зависимой таблице	`UNIQUE` на FK или FK = PK
1 : N	Пользователь — заказы	На стороне «много» (orders)	`NOT NULL`, если связь обязательна
N : 1	Сотрудники — отдел	На стороне «много» (employees)	Та же 1:N, другая точка зрения
M : N	Студенты — курсы	Оба FK в связующей таблице	`PRIMARY KEY (a_id, b_id)` или `UNIQUE`
Сама на себя	Сотрудник — руководитель	В той же таблице	`CHECK (manager_id <> id)`

05 Нормализация без боли

Каждый факт — в одном месте

Нормализация — это набор правил, которые избавляют от **дублирования** данных. Звучит академично, но суть одна: *каждый факт должен храниться ровно в одном месте*. Когда факт хранится в двух местах, рано или поздно они начнут противоречить друг другу.

Что не так с «одной большой таблицей»

orders_flat

order_id	customer	phone	product	price	qty
1	Анна	+7 900 111-22-33	Клавиатура	3500	1
1	Анна	+7 900 111-22-33	Мышь	1200	2
2	Анна	+7 900 111-22-33	Мышь	1200	1
3	Борис	+7 900 444-55-66	Монитор	18000	1

Так выглядит типичная таблица из Excel. Видишь, сколько раз повторяется Анна?

Аномалия	Что происходит
Обновления	Анна сменила телефон — менять в трёх строках. Забыл одну — в базе два разных телефона Анны, и какой верный, уже не узнать.
Вставки	Нельзя добавить новый товар в каталог, пока его кто-нибудь не купит: без заказа строку не создать.
Удаления	Удалили заказ №3 — и потеряли информацию о том, что Борис вообще существует и что мы продаём мониторы.

Решение — разложить таблицу по сущностям: `customers`, `products`, `orders`, `order_items`. Каждый факт (телефон Анны, цена мыши) хранится один раз, остальные на него ссылаются.

Три нормальные формы простыми словами

1

1НФ — в каждой ячейке одно значение

Никаких списков через запятую (`tags = 'sql,db'`) и повторяющихся групп столбцов (`phone1`, `phone2`, `phone3`). Много значений = отдельная таблица со связью 1:N.

2

2НФ — всё зависит от всего ключа

Актуально при составном ключе. В `order_items (order_id, product_id, quantity, product_name)` поле `product_name` зависит только от `product_id`, а не от всей пары — его место в `products`.

3

3НФ — ничего не зависит от «не ключа»

Неключевые столбцы зависят только от ключа, а не друг от друга. Если `department_name` определяется через `department_id` — это транзитивная зависимость, выносим отдел в свою таблицу.

```
-- ✗ department_name зависит от department_id, а не от сотрудника
employees (id, name, department_id, department_name)

-- ✓ Выносим отдел в свою таблицу
employees   (id, name, department_id)
departments (id, name)
```

→ Правило

Классическая формула 3НФ: **«ключ, весь ключ и ничего, кроме ключа»**. Каждый неключевой столбец должен описывать ключ (1НФ), весь ключ целиком (2НФ) и ничего, кроме ключа (3НФ). Для 95% задач 3НФ — достаточный и правильный уровень.

Денормализация: когда можно нарушать

Иногда дублирование оправдано — ради скорости чтения. Например, счётчик `comments_count` в посте, чтобы не считать комментарии при каждом показе ленты. Но это **осознанное** решение с понятной ценой: теперь ты сам отвечаешь за то, чтобы копия не разошлась с оригиналом.

- Сначала нормализуй. Денормализуй только когда **измерил** и увидел проблему.
- Запиши, где источник истины, а где копия, и как копия обновляется.
- Предпочитай инструменты базы: индексы, материализованные представления, генерируемые столбцы — прежде чем дублировать руками.

★ Важно понять

Снимок — это не дубль. Цена товара в позиции заказа (`order_items.price_cents`) выглядит как дублирование `products.price_cents`, но это **другой факт**: «по какой цене купили тогда», а не «сколько стоит сейчас». Если цена в каталоге изменится, старые заказы не должны пересчитаться. То же с адресом доставки в заказе. Это не нарушение нормализации, а правильное моделирование истории.

06 Как понять предметную область

От разговора с людьми до готовой схемы

Самые дорогие ошибки в схеме — не синтаксические, а смысловые: неправильно понятый бизнес. Если ты решил, что у заказа один адрес, а бизнес отправляет посылки на несколько, никакие индексы не помогут. Поэтому проектирование начинается **не с SQL**, а с понимания того, как устроен мир, который ты моделируешь.

Пошаговый процесс

1

Собери требования и истории

Поговори с теми, кто будет пользоваться системой, или внимательно прочитай ТЗ. Проси описать процесс словами: «Покупатель выбирает товары, оформляет заказ, оплачивает, ему отправляют посылку…»

2

Выпиши существительные и глаголы

Существительные — кандидаты в **сущности** (покупатель, товар, заказ). Глаголы — кандидаты в **связи** (оформляет, содержит, принадлежит). Детали и прилагательные — **атрибуты** (цена, дата, статус).

3

Составь словарь терминов

Договорись об одном слове для одного понятия. «Клиент», «пользователь», «покупатель» и «аккаунт» — это одно и то же или разные вещи? Ответ определяет, сколько будет таблиц.

4

Определи связи двумя вопросами

Для каждой пары сущностей: «сколько B у одного A?» и наоборот. Запиши ответы прямо на стрелках диаграммы.

5

Нарисуй ER-диаграмму

На бумаге, на доске, в dbdiagram.io или draw.io. Рисовать быстрее, чем писать SQL, а ошибки видны сразу. Перерисовать прямоугольник ничего не стоит, переделать таблицу с данными — дорого.

6

Заполни таблицы примерами

Придумай 3–5 реальных строк для каждой таблицы. Именно здесь вылезают проблемы: «а куда записать второй телефон?», «а если у товара нет категории?»

7

Проверь схему вопросами бизнеса

Выпиши 5–10 вопросов, на которые база должна отвечать («выручка по категориям за месяц», «кто не покупал 90 дней»), и мысленно напиши запросы. Если запрос получается мучительным — схема не та.

8

Ищи граничные случаи

Смена email, удаление пользователя, изменение цены, возврат товара, часовые пояса, однофамильцы, отменённый заказ. Каждый такой случай — вопрос к схеме.

Разбор на примере

Текст требований

«Покупатель оформляет заказ. В заказе может быть несколько товаров в разном количестве. Каждый товар относится к категории, категории бывают вложенными. Покупатель может оставить отзыв на купленный товар с оценкой от 1 до 5.»

Что нашли	Во что превращается
Существительные: покупатель, заказ, товар, категория, отзыв	Таблицы: `users`, `orders`, `products`, `categories`, `reviews`
«покупатель **оформляет** заказ»	1:N — `orders.user_id`
«в заказе **несколько** товаров в **разном количестве**»	M:N с атрибутом → сущность `order_items` с `quantity`
«товар **относится к** категории»	N:1 — `products.category_id`
«категории бывают **вложенными**»	Самоссылка — `categories.parent_id`
«отзыв на товар с оценкой **от 1 до 5**»	Две N:1 + `CHECK (rating BETWEEN 1 AND 5)`
Скрытый вопрос: «может ли один человек оставить два отзыва на товар?»	Уточняем у бизнеса → нет → `UNIQUE (user_id, product_id)`

Вопросы, которые стоит задавать всегда

Вопрос	На что влияет в схеме
Может ли у X быть несколько Y? А у Y несколько X?	Тип связи
Может ли X существовать без Y?	`NOT NULL` у внешнего ключа
Что должно произойти, когда X удаляют?	`ON DELETE`, мягкое удаление
Что здесь уникально? По чему ищут?	`UNIQUE`, индексы
Меняется ли это значение со временем? Нужна ли история?	Снимки, таблицы истории, `valid_from` / `valid_to`
Какие значения допустимы? Есть ли диапазоны?	`CHECK`, справочники
Какие отчёты и выборки нужны чаще всего?	Индексы, иногда денормализация
Сколько будет данных через год?	Типы ключей, партиционирование, индексы

Сущность или атрибут?

Частый вопрос: адрес — это столбец в `users` или отдельная таблица? Ответ зависит от предметной области. Помогает простой тест:

- Если у «штуки» **есть свои свойства** (у адреса — город, улица, индекс, комментарий курьеру) — скорее сущность.
- Если **их может быть несколько** у одного владельца (несколько адресов доставки) — точно сущность.
- Если на неё **ссылаются из разных мест** (адрес используют заказы и склады) — сущность.
- Если это одно простое значение, которое нужно только для отображения — атрибут.

> **✔ Лайфхак**
>
> Схема — живой документ. Не пытайся сделать идеально с первого раза: сделай хорошо для понятных сейчас требований, но выбирай решения, которые легко расширить. Добавить таблицу и связь просто, а вот разделить одну таблицу на две, когда в ней уже миллион строк, — больно.

07 Практика: интернет-магазин

Полная схема с объяснением решений

Соберём всё вместе. Требования — из разбора в предыдущей главе. Ниже диаграмма, полный SQL и объяснение, почему сделано именно так.

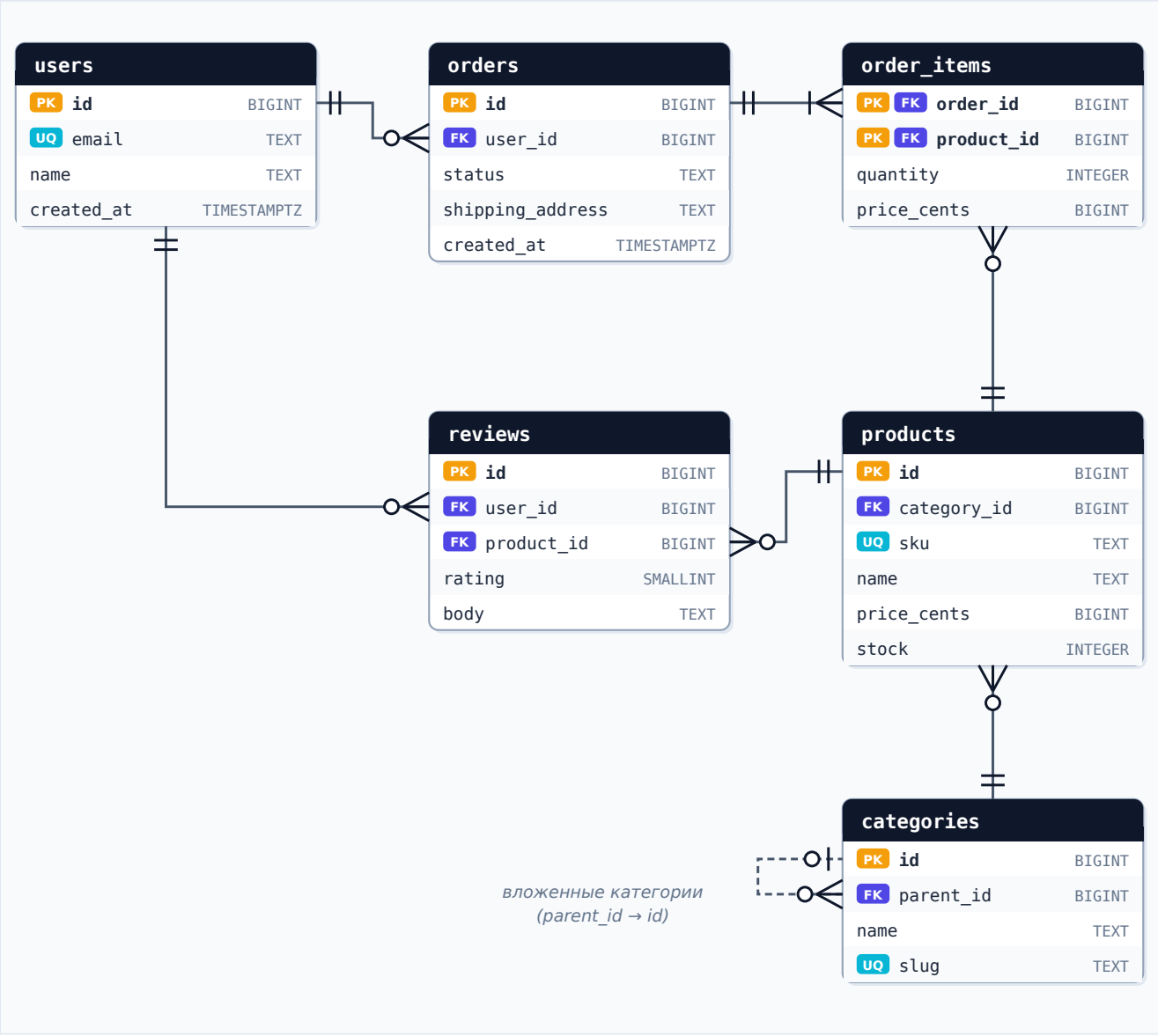


Схема интернет-магазина: шесть таблиц, все виды связей

Полный SQL

schema.sql

```sql
CREATE TABLE users (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email      TEXT        NOT NULL,
    name       TEXT        NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_uq ON users (lower(email));

CREATE TABLE categories (
    id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_id BIGINT REFERENCES categories (id) ON DELETE RESTRICT,
    name      TEXT NOT NULL,
    slug      TEXT NOT NULL UNIQUE,
    CONSTRAINT categories_not_own_parent CHECK (parent_id <> id)
);
CREATE INDEX categories_parent_id_idx ON categories (parent_id);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT  NOT NULL REFERENCES categories (id) ON DELETE RESTRICT,
    sku         TEXT    NOT NULL UNIQUE,
    name        TEXT    NOT NULL,
    price_cents BIGINT  NOT NULL CHECK (price_cents >= 0),
    stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
    is_active   BOOLEAN NOT NULL DEFAULT true,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX products_category_id_idx ON products (category_id);

CREATE TABLE orders (
    id               BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id          BIGINT NOT NULL REFERENCES users (id) ON DELETE RESTRICT,
    status           TEXT   NOT NULL DEFAULT 'new',
    shipping_address TEXT   NOT NULL,     -- снимок адреса на момент заказа
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT orders_status_chk CHECK (
        status IN ('new', 'paid', 'shipped', 'delivered', 'cancelled'))
);
CREATE INDEX orders_user_id_idx ON orders (user_id);

CREATE TABLE order_items (
    order_id    BIGINT  NOT NULL REFERENCES orders (id)   ON DELETE CASCADE,
    product_id  BIGINT  NOT NULL REFERENCES products (id) ON DELETE RESTRICT,
    quantity    INTEGER NOT NULL CHECK (quantity > 0),
    price_cents BIGINT  NOT NULL CHECK (price_cents >= 0), -- цена в момент покупки
    PRIMARY KEY (order_id, product_id)
);
CREATE INDEX order_items_product_id_idx ON order_items (product_id);

CREATE TABLE reviews (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT   NOT NULL REFERENCES users (id)    ON DELETE CASCADE,
    product_id BIGINT   NOT NULL REFERENCES products (id) ON DELETE CASCADE,
    rating     SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    body       TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT reviews_one_per_user UNIQUE (user_id, product_id)
);
CREATE INDEX reviews_product_id_idx ON reviews (product_id);
```

Почему сделано именно так

Решение	Зачем
`UNIQUE INDEX ... (lower(email))`	`Anna@Mail.ru` и `anna@mail.ru` — один и тот же человек. Регистр не должен создавать дубли.
Цена в копейках, `BIGINT`	Никаких ошибок округления float. 1999,90 ₽ хранится как `199990`.
`order_items.price_cents`	Снимок цены на момент покупки. Каталог может подорожать — старые заказы не изменятся.
`orders.shipping_address`	Снимок адреса. Пользователь переехал — история доставок остаётся правдивой.
Нет поля `orders.total`	Сумма вычисляется из позиций, поэтому никогда не разойдётся с ними. Если станет медленно — тогда и подумаем о кэше.
PK `(order_id, product_id)`	Один товар — одна строка в заказе. Купили две мыши — `quantity = 2`, а не две строки.
`order_items → orders`: CASCADE	Позиции — часть заказа и без него не живут.
`order_items → products`: RESTRICT	Товар, который кто-то покупал, удалять нельзя — его снимают с продажи через `is_active = false`.
`orders → users`: RESTRICT	Заказы — бухгалтерия и история. Пользователя с заказами не удаляем физически.
`reviews`: CASCADE + `UNIQUE (user_id, product_id)`	Отзывы не ценны без автора и товара; один отзыв на товар от человека.
`CHECK (parent_id <> id)`	Категория не может быть родителем самой себе.
Индекс на каждый FK	Быстрые JOIN и быстрые проверки при удалении родителя.

Проверяем схему запросами

```sql
-- Сумма каждого заказа (считаем, а не храним)
SELECT o.id, SUM(oi.quantity * oi.price_cents) AS total_cents
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
GROUP BY o.id;

-- Топ-5 товаров по среднему рейтингу (минимум 3 отзыва)
SELECT p.name, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(*) AS reviews
FROM products p
JOIN reviews r ON r.product_id = p.id
GROUP BY p.id, p.name
HAVING COUNT(*) >= 3
ORDER BY avg_rating DESC
LIMIT 5;
```

✓ Упражнение

Попробуй сам: добавь в эту схему **адреса пользователей** (у пользователя их несколько, один — основной), **купоны** (один купон на заказ, купон можно использовать ограниченное число раз) и **избранное** (пользователь — товары). Определи тип каждой связи двумя вопросами и реши, где будут FK и какие нужны ограничения.

08 Типы данных и именование

Мелочи, которые потом дорого исправлять

Какой тип выбрать

Что хранишь	Тип	Почему / чего избегать
Деньги	`BIGINT` (копейки) или `NUMERIC(12,2)`	Никогда `FLOAT`/`REAL`: 0.1 + 0.2 = 0.30000000000000004. Валюту храни рядом, если их несколько
Момент времени	`TIMESTAMPTZ`	Хранится в UTC, без путаницы с поясами. Не `TIMESTAMP` без пояса и тем более не `TEXT`
Только дата	`DATE`	День рождения, дата договора — без времени
Строки	`TEXT` (+ `CHECK` на длину при необходимости)	В PostgreSQL `VARCHAR(n)` не быстрее `TEXT`. В MySQL — `VARCHAR(n)`
Да / нет	`BOOLEAN NOT NULL DEFAULT false`	Не `'Y'/'N'` и не `0/1` в INTEGER
Телефон, индекс, ИНН, номер карты	`TEXT`	Это не числа: ведущие нули, «+», дефисы. С ними не делают арифметику
Email	`TEXT` + уникальный индекс по `lower(email)`	Или тип `CITEXT` (регистронезависимый текст)
Идентификаторы	`BIGINT` или `UUID`	`INTEGER` может закончиться (~2,1 млрд) — экономия 4 байт не стоит миграции на живом проекте
Статус	`TEXT` + `CHECK`, `ENUM` или справочник	См. главу 4
Гибкие данные	`JSONB`	Только для действительно произвольной структуры (настройки, ответы внешних API). Не замена нормальным столбцам
Возраст	Не храни! Храни `birth_date`	Возраст меняется каждый день, дата рождения — никогда

⚠ Осторожно

Правило для любых вычисляемых значений: если значение можно **надёжно** вычислить из других данных (возраст, сумма заказа, количество позиций) — по умолчанию вычисляй, а не храни. Хранимая копия — это ещё одно место, которое может разойтись с правдой.

Правила именования

- **snake_case** для всего: `order_items`, `created_at`. Не `OrderItems`, не `createdAt` — в PostgreSQL имена без кавычек приводятся к нижнему регистру.
- **Таблицы — во множественном числе** (`users`, `orders`) или в единственном — выбери одно и держись этого везде. Множественное удобно ещё и тем, что `user` и `order` — зарезервированные слова.
- **Первичный ключ —** `id`, внешний — `<сущность>_id`: `user_id`, `category_id`. Сразу видно, куда ведёт ссылка.
- **Если ссылок на одну таблицу несколько — называй по роли:** `author_id`, `assignee_id`, `sender_id`, `recipient_id`.
- **Булевы поля — с** `is_` **/** `has_`**:** `is_active`, `has_discount`. Читается как вопрос.
- **Время — с** `_at`**, даты — с** `_on` **или** `_date`**:** `created_at`, `paid_at`, `birth_date`.
- **Единицы измерения — в имени:** `price_cents`, `weight_grams`, `duration_seconds`. Никто не будет гадать, рубли это или копейки.
- **Без сокращений и транслита:** `quantity`, а не `qty` или `kolvo`.
- **Ограничения и индексы — по шаблону:** `orders_user_id_fkey`, `orders_user_id_idx`, `products_sku_uq`.

Плохо	Хорошо	Почему
`tbl_user`, `UserTable`	`users`	Префиксы и CamelCase — лишний шум
`date`, `type`, `order`	`created_at`, `kind`, `orders`	Зарезервированные слова
`price` **(в чём?)**	`price_cents`	Единица в имени
`active INT`	`is_active BOOLEAN`	Правильный тип и имя-вопрос
`user1_id`, `user2_id`	`buyer_id`, `seller_id`	Имя по роли
`data`, `info`, `value`	Конкретное название	Ничего не говорит о содержимом

09 Индексы: необходимый минимум

Чтобы схема не только была верной, но и быстрой

Индекс — это как алфавитный указатель в конце книги: вместо того чтобы листать все 500 страниц, ты сразу открываешь нужную. Без индекса база при поиске читает таблицу целиком (*sequential scan*). На тысяче строк это незаметно, на десяти миллионах — катастрофа.

Где индексы появляются сами, а где нет

Объект	Индекс создаётся автоматически?
`PRIMARY KEY`	✓ Да
`UNIQUE`	✓ Да
`FOREIGN KEY`	✗ **Нет** (в PostgreSQL) — создавай сам
Столбцы в `WHERE`, `ORDER BY`, `JOIN`	✗ Нет — по необходимости

Составные индексы и правило «левого края»

Индекс по `(a, b)` отсортирован сначала по `a`, внутри — по `b`. Поэтому он помогает запросам с условием на `a` и на `a + b`, но **не помогает** запросу только по `b` — как телефонный справочник, отсортированный по фамилии, бесполезен для поиска по имени.

Запрос	Индекс (user_id, created_at) поможет?
`WHERE user_id = 42`	✓ Да
`WHERE user_id = 42 ORDER BY created_at DESC`	✓ Да, и сортировка бесплатная
`WHERE created_at > '2026-01-01'`	✗ Нет — нужен свой индекс по created_at

```
-- Частый запрос: последние заказы пользователя
SELECT id, status, created_at
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;

-- Индекс под него: сначала фильтр, потом сортировка
CREATE INDEX orders_user_created_idx ON orders (user_id, created_at DESC);

-- Смотрим, как база выполняет запрос
EXPLAIN ANALYZE
SELECT id, status, created_at
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Сколько индексов нужно

- **Каждый FK** — почти всегда.
- **Столбцы частых фильтров и сортировок** — по результатам реальных запросов.
- **Не индексируй всё подряд.** Каждый индекс замедляет `INSERT` / `UPDATE` / `DELETE` и занимает место. Индекс на столбце `is_active`, где 99% строк `true`, почти бесполезен.
- **Проверяй, а не угадывай:** `EXPLAIN ANALYZE` покажет, использует ли база индекс и сколько времени занимает каждый шаг.

> **✔ Лайфхак**
>
> Если видишь в плане `Seq Scan` на большой таблице в частом запросе — это первый кандидат на новый индекс. Если видишь `Index Scan` или `Index Only Scan` — индекс работает.

10 Лайфхаки и антипаттерны

Опыт, за который обычно платят продакшеном

Лайфхаки

1 `created_at` и `updated_at` — в каждой таблице

Стоит копейки, а при разборе инцидентов, аналитике и отладке выручает постоянно. «Когда появилась эта строка?» — вопрос, который рано или поздно задают про любую таблицу.

2 Храни снимки того, что меняется

Цена в заказе, адрес доставки, название тарифа на момент подписки. Историю нельзя восстановить задним числом.

3 Деньги — в целых числах

Копейки в `BIGINT` (или `NUMERIC`). Float для денег — гарантированные расхождения в отчётах.

4 Всё время — в UTC (`TIMESTAMPTZ`)

Переводи в часовой пояс пользователя только при показе. Смешение поясов в базе — одна из самых неприятных ошибок для отладки.

5 Начинай со строгих ограничений

`NOT NULL`, `CHECK`, `UNIQUE` проще ослабить потом, чем добавить: на таблицу, в которой уже лежат «грязные» данные, ограничение просто не встанет, пока не почистишь.

6 Мягкое удаление — осознанно

`deleted_at TIMESTAMPTZ` сохраняет историю, но **каждый** запрос теперь должен помнить `WHERE deleted_at IS NULL`, а уникальность — делаться частичным индексом. Используй там, где история действительно нужна.

7 Каждое изменение схемы — через миграции

Версионированные SQL-файлы в репозитории, применяемые инструментом миграций. Никаких «я быстренько поправил на проде руками».

8 Ломающие изменения — в несколько шагов

Переименование или удаление столбца на живой системе делай по схеме expand → contract (пример ниже).

9 Держи диаграмму рядом с кодом

ER-диаграмма в репозитории (DBML для dbdiagram.io, Mermaid, или скриншот) — лучший онбординг для нового человека в команде.

10 **Тестовые данные, похожие на правду**

Скрипт с реалистичными данными: длинные имена, эмодзи, пустые необязательные поля, тысячи строк. Многие проблемы видны только на «живых» данных.

11 **Связанные изменения — в транзакции**

Создание заказа и его позиций, списание остатка — либо всё, либо ничего (`BEGIN … COMMIT`). Иначе при сбое останется заказ без позиций.

Безопасное переименование столбца

```sql
-- Переименование колонки без простоя (expand -> contract)
-- 1. Добавляем новую колонку
ALTER TABLE users ADD COLUMN full_name TEXT;
-- 2. Приложение пишет в обе колонки (новая версия кода)
-- 3. Переносим старые данные
UPDATE users SET full_name = name WHERE full_name IS NULL;
-- 4. Приложение читает только full_name
-- 5. Когда старая колонка никому не нужна — удаляем
ALTER TABLE users DROP COLUMN name;
```

Антипаттерны

Антипаттерн	Чем плох	Как правильно
Списки в одной ячейке (`'1,5,7'`)	Нет FK, медленный поиск, сложное обновление	Отдельная таблица (1:N или M:N)
`phone1`, `phone2`, `phone3`	А если нужен четвёртый? Поиск по всем столбцам сразу	Таблица `user_phones`
Нет внешних ключей «для скорости»	Осиротевшие строки и мусор гарантированы	FK + индекс на него
`FLOAT` **для денег**	Ошибки округления	`BIGINT` в копейках или `NUMERIC`
Магические значения (`-1`, `'1970-01-01'`)	Вместо «нет значения» — фальшивое значение, которое попадает в отчёты	`NULL` или отдельный столбец-статус
«God table» на 80 столбцов	Половина пустая, непонятно, что к чему	Разбить по сущностям, редкое — в 1:1
EAV-таблица атрибутов	Нет типов, ограничений, FK; запросы — ад	Обычные столбцы или `JSONB`
Полиморфная ссылка (`type` + `id`)	Невозможен FK — база не проверяет ссылку	Явные FK + `CHECK` (ниже)
Хранить вычисляемое без нужды	Расходится с источником	Вычислять в запросе

```sql
-- ✗ EAV: "универсальная" таблица атрибутов
CREATE TABLE attributes (
    entity_id BIGINT,
    name      TEXT,     -- 'price', 'color', 'weight'
    value     TEXT      -- всё строкой: ни типов, ни CHECK, ни FK
);
```

```
-- ✗ Полиморфная связь: база не может проверить ссылку
CREATE TABLE comments (
    id               BIGINT PRIMARY KEY,
    commentable_type TEXT,     -- 'post' или 'photo'
    commentable_id   BIGINT,   -- FK невозможен: неизвестно, на какую таблицу
    body             TEXT
);
```

```
-- ✓ Явные внешние ключи + правило "ровно одна ссылка"
CREATE TABLE comments (
    id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    post_id  BIGINT REFERENCES posts (id)  ON DELETE CASCADE,
    photo_id BIGINT REFERENCES photos (id) ON DELETE CASCADE,
    body     TEXT NOT NULL,
    CONSTRAINT comments_one_target CHECK (num_nonnulls(post_id, photo_id) = 1)
);
```

11 Чек-лист и шпаргалка

Пробеги глазами перед каждой новой таблицей

Чек-лист схемы

Ключи

- ☐ У каждой таблицы есть первичный ключ.
- ☐ Натуральные уникальные значения (email, артикул, slug) защищены `UNIQUE`.
- ☐ Каждая ссылка на другую таблицу оформлена как `FOREIGN KEY`.
- ☐ Для каждого FK осознанно выбрано `ON DELETE`.
- ☐ На каждом FK есть индекс.

Связи

- ☐ Для каждой пары сущностей тип связи определён двумя вопросами.
- ☐ FK находится на стороне «много».
- ☐ У 1:1 есть `UNIQUE` (или общий PK) — кардинальность гарантирована схемой.
- ☐ Каждая M:N реализована через связующую таблицу с защитой от дублей.
- ☐ Обязательные связи — `NOT NULL`.

Данные

- ☐ Все столбцы `NOT NULL`, кроме тех, где отсутствие значения что-то означает.
- ☐ Бизнес-правила (диапазоны, статусы, даты) закреплены через `CHECK`.
- ☐ Деньги — не float, время — `TIMESTAMPTZ`, телефоны и индексы — `TEXT`.
- ☐ Нет списков в ячейках и пронумерованных столбцов.
- ☐ Каждый факт хранится в одном месте; снимки (цена, адрес) — осознанно.
- ☐ Есть `created_at` (и `updated_at`, если строки меняются).

Смысл

- ☐ Схема отвечает на все вопросы бизнеса из списка требований.
- ☐ Граничные случаи (удаление, смена данных, история) продуманы.
- ☐ Имена понятны человеку, который видит схему впервые.
- ☐ ER-диаграмма актуальна и лежит рядом с кодом.

Шпаргалка на одну страницу

Задача	Решение
Идентифицировать строку	`id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY`
Значение не должно повторяться	`UNIQUE` (или уникальный индекс по выражению)
Значение обязательно	`NOT NULL`
Значение в диапазоне / из списка	`CHECK (...)`
Ссылка на другую таблицу	`REFERENCES t (id)` + индекс на столбец
1 : 1	FK + `UNIQUE`, или FK = PK
1 : N / N : 1	FK на стороне «много»
M : N	Связующая таблица, `PRIMARY KEY (a_id, b_id)` + индекс по `b_id`
Дерево / иерархия	`parent_id REFERENCES` та же таблица
Удалить детей вместе с родителем	`ON DELETE CASCADE` (только для «частей целого»)
Запретить удаление родителя	`ON DELETE RESTRICT` / по умолчанию
Ссылка может исчезнуть	`ON DELETE SET NULL` + nullable FK
Email без учёта регистра	`CREATE UNIQUE INDEX ... (lower(email))`
Уникально среди неудалённых	Частичный индекс `... WHERE deleted_at IS NULL`
Нет пересечений по времени	`EXCLUDE USING gist (... WITH &&)`
Деньги	`BIGINT` в копейках
Время	`TIMESTAMPTZ`, всё в UTC
Цена / адрес на момент события	Снимок в строке события
Медленный запрос	`EXPLAIN ANALYZE` → индекс под фильтр и сортировку

★ Важно понять

Главное из всего гайда в трёх фразах. Сначала пойми предметную область, потом рисуй, потом пиши SQL. Каждый факт храни в одном месте и связывай таблицы внешними ключами. Пусть база сама охраняет правила — ограничения дешевле любых багов.